

ACTUATOR 26 // MODULE 05

GYRO GESTURE CONTROLLED BOT



| 6-Axis Motion Tracking & Master-Slave Dual Microcontroller Protocol

MASTER: ESP8266 + MPU-6050

SLAVE: ESP32 + TB6612FNG

COMM: WI-FI TCP SOCKET (PORT 5000)

SYSTEM ARCHITECTURE

Master-Slave Wireless Topology

A dual-node embedded system transmitting real-time 6-axis gesture telemetry over a dedicated local Wi-Fi TCP socket network.

NODE 1 // GESTURE MASTER

ESP8266 (NodeMCU) + MPU-6050

Worn on the user's hand. Polls 3-axis accelerometer values (a_x , a_y , a_z) over I2C (SCL=D1, SDA=D2). Hosts Wi-Fi Access Point **ROBOCEK-GESTURE** and runs TCP Server on port 5000.

NODE 2 // ROBOT SLAVE

ESP32 + TB6612FNG Driver

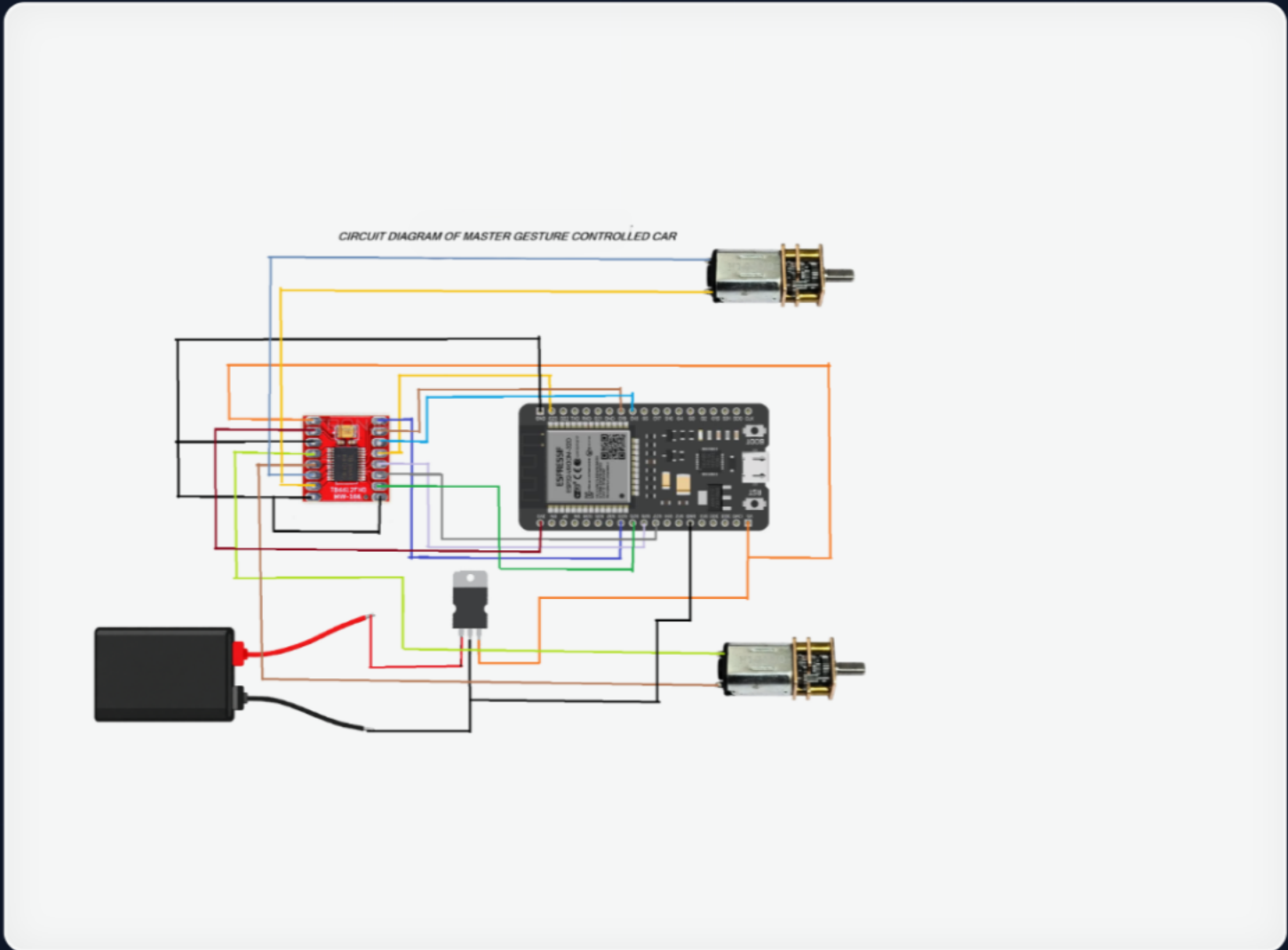
Onboard vehicle controller. Connects to Master Access Point as Wi-Fi Station (**192.168.4.1**). Reads incoming TCP gesture command bytes and drives dual DC motors via PWM.

HARDWARE INTERACTION

Master & Slave Circuit Diagrams

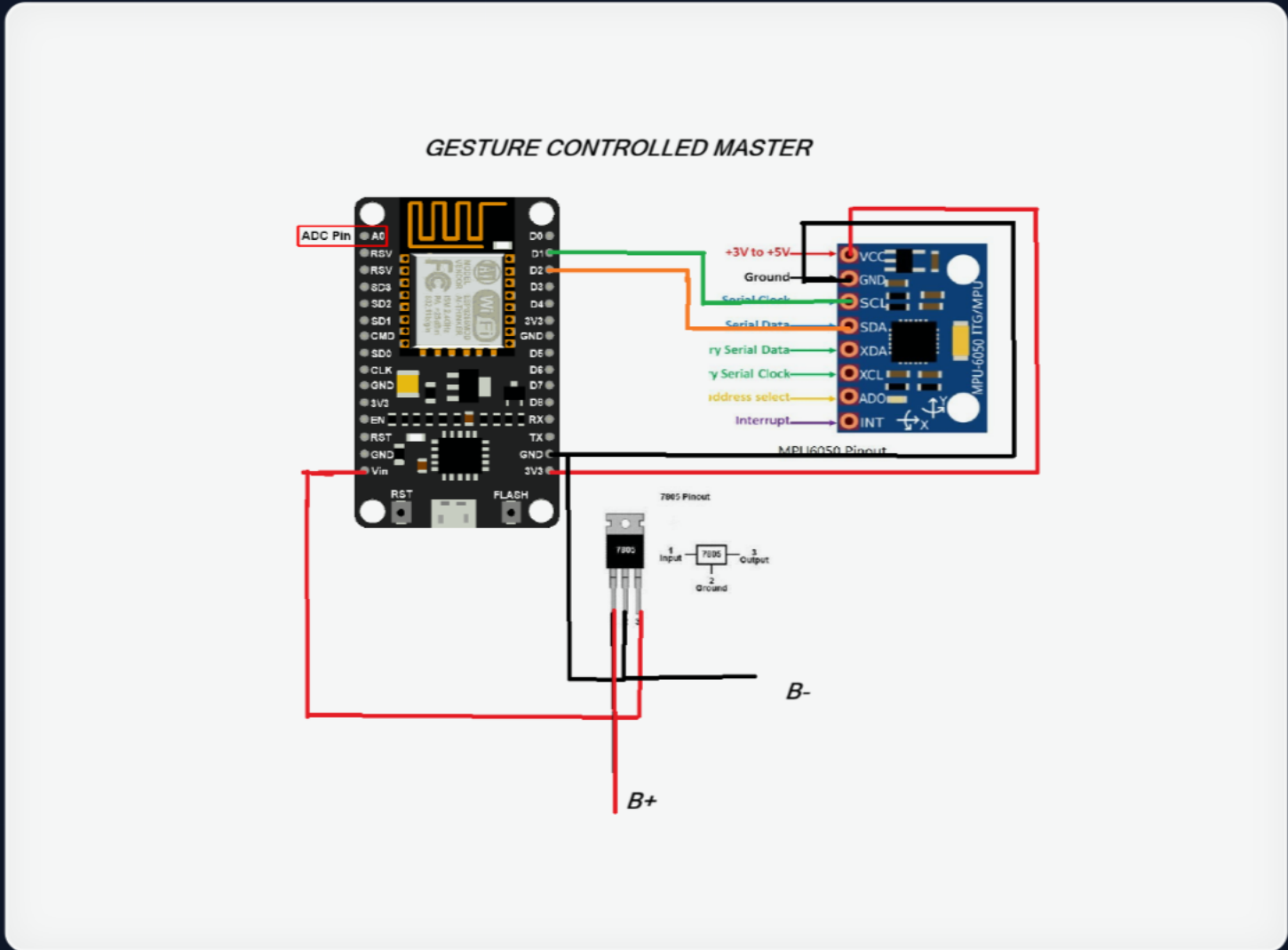
Complete hardware schematics for the hand-worn Master Controller (ESP8266 + MPU-6050) and vehicle Slave Chassis (ESP32 + TB6612FNG).

NODE 1 // MASTER GESTURE CONTROLLER



ESP8266 NodeMCU SoftAP TCP Server (Port 5000)
MPU-6050 Gyro/Accel SDA: D2 (GPIO4) | SCL: D1 (GPIO5)
LM7805 Voltage Regulator 5V Vin Logic Power

NODE 2 // SLAVE ROBOT CAR



ESP32 Wi-Fi STA TCP Client
Microcontroller (192.168.4.1)
TB6612FNG STBY: 23 | PWM: 33/25 | IN:
Driver 18/19/26/27
Dual N20 Motors & 7805 Battery Powered Motor Rail

SENSOR MECHANICS

MPU-6050 6-Axis Motion Tracking

The MPU-6050 combines a 3-axis MEMS accelerometer and 3-axis gyroscope with an internal Digital Motion Processor (DMP) communicating over I2C.

- ↳ **3-Axis Accelerometer**
Measures gravitational acceleration ($g = 9.81 \text{ m/s}^2$). Tilt changes component vectors a_x, a_y, a_z .
- ↳ **Digital Low-Pass Filter**
Set to 21 Hz bandwidth to smooth out motor noise and hand tremors.
- ↳ **Sensitivity Range Configuration**
Configured for $\pm 8g$ accelerometer range and $\pm 500^\circ/\text{s}$ gyro range.

```
// Master setup initialization
Wire.begin(MPU_SDA, MPU_SCL); // SDA=D2, SCL=D1

if (!mpu.begin()) {
  Serial.println("MPU6050 NOT FOUND!");
  while (1) delay(1000);
}

mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
mpu.setGyroRange(MPU6050_RANGE_500_DEG);
mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
```

MASTER FIRMWARE

Wi-Fi SoftAP & TCP Server Setup

The ESP8266 master configures itself as an autonomous Access Point creating an isolated wireless control network.

```
const char *AP_SSID = "ROBOCEK-GESTURE";
const char *AP_PASSWORD = "12345678";

WiFiServer server(5000);
WiFiClient client;

void setup() {
  Serial.begin(115200);
  // ... MPU6050 initialization ...

  WiFi.mode(WIFI_AP);
  WiFi.softAP(AP_SSID, AP_PASSWORD);

  Serial.print("Access Point IP: ");
  Serial.println(WiFi.softAPIP()); // 192.168.4.1

  server.begin(); // Start listening on TCP Port 5000
}
```

GESTURE CLASSIFICATION

Tilt Thresholding Logic

Hand orientation is decoded by measuring static tilt acceleration vectors along X and Y axes.

GESTURE THRESHOLD EQUATIONS (M/S²)

Tilt Forward: $a_x > +3.0 \Rightarrow 'F'$ | Tilt Backward: $a_x < -3.0 \Rightarrow 'B'$

Tilt Left: $a_y > +3.0 \Rightarrow 'L'$ | Tilt Right: $a_y < -3.0 \Rightarrow 'R'$

```
#define TILT_FORWARD 3.0
#define TILT_BACKWARD -3.0
#define TILT_LEFT 3.0
#define TILT_RIGHT -3.0

char command = 'S';

if (ax > TILT_FORWARD)      command = 'F';
else if (ax < TILT_BACKWARD) command = 'B';
else if (ay > TILT_LEFT)    command = 'L';
else if (ay < TILT_RIGHT)   command = 'R';
else                        command = 'S';
```

MASTER FIRMWARE

Telemetry Transmission Loop

Continuously polls the MPU-6050, evaluates gesture tilt state, and writes single-byte packets over TCP socket.

```
void loop() {  
    // Re-accept client if connection drops  
    if (!client || !client.connected()) {  
        client = server.available();  
        if (client) Serial.println("SLAVE CONNECTED");  
        delay(10);  
        return;  
    }  
  
    sensors_event_t a, g, temp;  
    mpu.getEvent(&a, &g, &temp);  
  
    char command = evaluateGesture(a.acceleration.x, a.acceleration.y);  
  
    // Send 1-byte command to slave car  
    client.write(command);  
    delay(50); // 20 Hz update rate  
}
```

PROTOCOL SPECIFICATION

Master-Slave Telemetry Truth Table

Complete mapping of hand tilt posture to transmitted TCP byte and slave motor execution state.

GESTURE POSTURE	ACCEL VECTOR (M/S ²)	TCP BYTE	SLAVE MOTOR STATE	LEFT / RIGHT MOTOR PWM
Hand Tilted Forward	$a_x > +3.0$	'F'	Both Motors Forward	+200 / +200
Hand Tilted Backward	$a_x < -3.0$	'B'	Both Motors Reverse	-200 / -200
Hand Tilted Left	$a_y > +3.0$	'L'	Spin Turn Left	-200 / +200
Hand Tilted Right	$a_y < -3.0$	'R'	Spin Turn Right	+200 / -200
Hand Flat / Level	$-3.0 \leq a_{x,y} \leq +3.0$	'S'	All Motors Stopped	0 / 0

SLAVE FIRMWARE

ESP32 Station & TCP Reconnection

The ESP32 vehicle automatically joins the Master AP and maintains an active TCP connection to port 5000.

```
void connectToMaster() {
    Serial.println("Connecting to MASTER...");
    WiFi.mode(WIFI_STA);
    WiFi.begin("ROBOCEK-GESTURE", "12345678");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    // Connect to Master TCP Socket Server
    while (!client.connect("192.168.4.1", 5000)) {
        Serial.println("TCP connection failed");
        delay(1000);
    }
    Serial.println("MASTER LINK ESTABLISHED");
}
```

SLAVE FIRMWARE

TB6612FNG Differential Motor Control

Bidirectional PWM speed control helper functions driving dual DC motors via the TB6612 H-Bridge driver.

ESP32 Pin Mapping

RC_MOTOR_STBY	GPIO 23 (HIGH)
RC_LEFT_PWM	GPIO 33
RC_LEFT_IN1 / IN2	GPIO 18 / 19
RC_RIGHT_PWM	GPIO 25
RC_RIGHT_IN1 / IN2	GPIO 26 / 27

```
void leftMotor(int speed) {  
    if (speed > 0) {  
        digitalWrite(RC_LEFT_IN1, HIGH);  
        digitalWrite(RC_LEFT_IN2, LOW);  
    } else if (speed < 0) {  
        digitalWrite(RC_LEFT_IN1, LOW);  
        digitalWrite(RC_LEFT_IN2, HIGH);  
    } else {  
        digitalWrite(RC_LEFT_IN1, LOW);  
        digitalWrite(RC_LEFT_IN2, LOW);  
    }  
    analogWrite(RC_LEFT_PWM, abs(speed));  
}
```

SLAVE FIRMWARE

TCP Socket Dispatch & Auto-Stop

Dispatches incoming commands in real-time and triggers immediate motor shutdown if wireless link drops.

```
void loop() {  
    // Fail-safe reconnect if Wi-Fi or TCP disconnects  
    if (WiFi.status() != WL_CONNECTED || !client.connected()) {  
        stopCar();  
        connectToMaster();  
        return;  
    }  
  
    if (client.available()) {  
        char command = client.read();  
        switch (command) {  
            case 'F': forward();    break;  
            case 'B': backward();  break;  
            case 'L': turnLeft();  break;  
            case 'R': turnRight(); break;  
            case 'S': stopCar();    break;  
            default: stopCar();     break;  
        }  
    }  
}
```

DIAGNOSTIC & CHECKLIST

Field Calibration & Testing

MPU-6050 Calibration

Place hand controller flat on level table before boot to verify $a_x \approx 0, a_y \approx 0, a_z \approx 9.8 \text{ m/s}^2$.

Wireless Socket Ping

Keep transmission delay at 50ms (20 Hz update rate) to ensure rapid response without socket buffer overflow.

Power Supply Separation

Power ESP32 logic and TB6612 VM motor power separately to prevent micro-reset on high motor turn-on spikes.

Fail-Safe Auto-Stop

Always verify vehicle stops immediately when hand controller is powered off or moves out of AP range.

ACTUATOR
26
GESTURE
BOT
PASSED